You must complete all sections clearly, honestly, and using observable evidence.
This document is a release-readiness systems audit.

Section 1 – Sprint Outcome & Release Readiness

- Sprint Goal (copy exactly from your Sprint 5 Plan):
  By the end of Sprint 5, a user will be able to view and filter available bike trails by difficulty and surface type; see the money savings of choosing to bike over other transportation modes; and log rides to their activity log with a correctly stored date and no ability to log rides in the future.
- Sprint Goal Met:
  Partially
- In ONE sentence, what can a user now do?
  (Must match your demo exactly. No vague descriptions)
  A user can now view and filter available bike trails by difficulty and surface type, see the money AND CO2 savings of choosing to bike over other transportation modes, and log rides to their activity log with a correctly stored date and no ability to log rides in the future.
  During your demo, did your system behave consistently across repeated use?
  No
- What specifically failed, became unstable, or required explanation?
  (Include crashes, inconsistent behavior, missing persistence, timing issues, unreliable workflows, dependency failures, etc.)
  Trail Browsing was the failure point. Trail cards rendered on first load and the difficulty filter worked on the initial pass, but switching Easy -> Moderate -> Hard and then back to Easy on the second consecutive set of switches left previously filtered cards stale until the page was hard-refreshed. The trail detail modal also did not consistently close on outside-click, it closed for the first trail demoed but not the third. The Activity Log Fix and Money Savings Display both ran cleanly across repeated demo runs.
- Did any part of your demo rely on:
  (Check all that apply)
  ☐ A perfect path
  ☐ Pre-configured data/state
  ☒☒ Explanation instead of demonstration
  ☐ Hidden setup
  ☒ Restarting/retrying
  ☐ “It works if…” conditions
  ☒ Q&A to complete missing workflow
  ☐ None of the above
- What still prevents your system from being considered release-ready?
  (Be specific about remaining instability, workflow gaps, technical debt, integration risk, testing weakness, or incomplete systems.)
  Trail Browsing’s filter state management is fragile, consecutive filter changes do not always trigger a clean re-render of the trail list. The trail detail modal’s outside-click

close handler is inconsistent across different trail cards. Heading into the feature-freeze sprint, 9 deferred Feature tickets (#6–#15, excluding #14) are still visible in the open GitHub backlog, creating scope pressure against the stabilization work the existing Trail Browsing flow still needs.

Section 2 – Structural Failure (Evidence Required)

- Primary Structural Issue (choose one):
  Weak retrospective enforcement
- Describe exactly what happened:
  (Describe the actual failure behavior – not intentions.)
  During Yohaan's end-to-end demo walkthrough at Sprint Review on 5/15, the team ran the Trail Browsing flow twice end-to-end in the same browser session. The first run worked: navigating to /trails, selecting Easy difficulty, and clicking a trail card all behaved correctly. The second run, immediately after on the same session, failed, switching Easy -> Moderate -> Hard left Easy-difficulty trails still visible until a manual refresh. Clicking outside the trail detail modal on a different trail also failed to close it on the third attempted close. The Activity Log Fix and Money Savings Display passed both runs.
- Evidence 1 (REQUIRED – must be observable):
  Demo behavior
  Explain:
  Two consecutive Trail Browsing demo runs produced different filter behavior on the same browser session. The first run filtered correctly; the second run left stale trail cards visible after consecutive filter changes. The trail detail modal's outside-click close behavior also varied across attempts, closing reliably on the first attempt, failing on the third. Both observed live in front of the class during Sprint Review.
- Evidence 2 (REQUIRED – different from above):
  GitHub activity
  Explain:
  The GitHub Issues tab shows 9 open Feature tickets at Sprint 5 close (#6 Environmentally friendly organization of shops, #7 Forum to talk, #8 See Routes on Map, #9 Partnerships option with shops, #10 Favorite Routes, #11 Personalized recs, #12 Gamify app, #13 Showcase type of bike, #15 Customize Routes for fitness levels) against 47 closed issues. Every open ticket is labeled Feature, zero bug or chore tickets remain, meaning the residual backlog is entirely unfinished functional scope. All 9 were opened on March 4 by one contributor and have not been refined since project setup. Even though Sprint 5 intentionally deferred these per the plan, they remain visible going into the feature-freeze sprint.
- Evidence 3 (REQUIRED – Stability & Repeatability Evidence):
  What type of repeatability evidence best reflects your system behavior?
  Demo-only stability (worked once but not repeatedly)
  Explain:
  (Must reference observable demo behavior, testing results, GitHub activity, or workflow

evidence.)
Trail Browsing passed Yohaan's walkthrough check on Dev Day 5 on a single end-to-end run and was moved to Done. The walkthrough was actually run by the team a couple minutes late rather than during the final 10 minutes of class as the Sprint 4 rule specified, and it never re-exercised features on a second consecutive run within the same session. Sprint Review on 5/15 was the first time Trail Browsing was run twice back-to-back on the same browser session, and the second run surfaced the filter stale-state bug no one had previously seen.

Section 3 – Root Cause Analysis (System-Level Only)

- Complete this sentence:
  Our system allowed instability because…
  the Sprint 4 retrospective rule was executed at a different time than its written wording specified, "five minutes after class" rather than "the final ten minutes of every class period", which moved the walkthrough out of observable in-class workflow and made it trivially skippable when class ended in a rush or the whole team was absent (5/13 and 5/14), and the rule's single-pass walkthrough never required a second back-to-back run to expose features that worked once but failed on repeat.
- Explain the REAL system failure clearly:
  (No blaming individuals. Focus on workflow, timing, validation, enforcement, technical debt, integration structure, or Definition of Done failures.)
  Two structural gaps stacked. First, the Sprint 4 rule was written as "during the final 10 minutes of every class period" but the Sprint 5 planning document committed to running it "5 minutes after class every day." That timing shift removed the in-class observability of the rule and made it skippable when class ended abruptly or members had to leave immediately. Second, the rule only required one successful end-to-end run; it never required a second back-to-back run on the same browser session, so filter-state bugs that only appear on consecutive interactions had no scheduled chance to surface. Compounding both gaps, the Sprint 5 absence calendar lost all four team members on 5/13 and 5/14, the two class periods directly before the 5/15 demo, eliminating any buffer for a second pass to catch what the first pass missed.
- Which of these system behaviors allowed the issue to continue?
  (Check all that apply)
  ☐ Testing happened too late
  ☒ Work reached Done without validation
  ☐ Technical debt was ignored
  ☐ Integration happened too late
  ☐ Workflow discipline weakened under pressure
  ☒ Retrospective rule was not enforced
  ☒ Scope exceeded team capacity
  ☒ Demo readiness was assumed instead of validated
  ☒ Stability was never tested repeatedly

☒ Team depended on explanation instead of reliability
☐ Other

- Why did your system fail to detect this BEFORE demo day?
  (This is REQUIRED and should focus on system weaknesses, not effort.)
  The walkthrough was executed as a single-pass check after class with no requirement to repeat the flow on the same session. A filter-state bug that only surfaces on the second consecutive interaction will never be triggered by a single linear walkthrough. There is also no scheduled multi-feature integration run that exercises all completed Sprint Goal features back-to-back twice on the merged main branch, so cross-feature and repeat-use regressions have no place to surface in the workflow before they reach the live demo.

Section 4 – Sprint 5 Rule Evaluation (From Sprint 4 Retrospective Change Plan)

- Sprint 4 Retrospective Rule (copy exact wording from your Sprint 4 Change Plan)
  At the final 10 minutes of every class period beginning Dev Day 3, Yohaan (Demo Verifier) will run an unbroken end-to-end walkthrough of every completed Sprint Goal component on a clean, logged out browser, and any feature that fails, requires verbal explanation, or behaves inconsistently must be moved back to "In Progress" on the GitHub board before class ends.
- Was this rule followed consistently during Sprint 5?
  Sometimes
- **WHO** was responsible for enforcing this rule?
  Name + Role: **Sathkrith**
- Did this person actually enforce the rule during Sprint 5?
  Yes

Break Point Analysis

- At what exact point did the rule fail OR become ineffective?
  (Be specific: development day, workflow moment, testing stage, integration point, demo prep, etc.)
  The rule was structurally broken from the start of Sprint 5: the written wording said "final 10 minutes of every class period" but we decided to run the test at home most days. That moved the walkthrough out of observable class workflow before Sprint 5 even began. The concrete failure point was Dev Day 5, when Trail Browsing passed a single after-class walkthrough run and was marked Done. On 5/13 and 5/14 — the final two class periods before the 5/15 demo — three members were absent and no walkthrough ran at all.
- What **SHOULD** have happened instead?
  The walkthrough should have run inside class time, during the final 10 minutes as the rule was originally written, so that absences and post-class scheduling could not displace it. It should have required at least two consecutive end-to-end runs in the same browser session before any feature could be marked as Done.

- What evidence shows whether this rule actually changed team behavior?
  (Must reference observable workflow behavior)
  However, all three runs happened 5 minutes after class rather than in the final 10 minutes of class, and on 5/13 and 5/14 no walkthrough ran at all due to majority of the team being absent. Those two missed days are the inflection point between Sprint 4's structural improvement and Sprint 5's demo failure.
- Did this rule actively prevent regression during Sprint 5?
  Partially

Explain:

It prevented Sprint 4's specific date-offset bug from recurring, which was the rule's primary intended target. the Activity Log Fix passed both demo runs cleanly. It did not prevent the new Trail Browsing failure because (a) the rule's actual execution time drifted to after class, removing in class observability and consequence, and (b) the rule only required a single successful pass, not a second consecutive run on the same session

---

Section 5 – Technical Debt & Integration Readiness

- What technical debt still threatens system reliability?
  (Examples: duplicated code, fragile logic, missing validation, poor architecture, rushed fixes, weak testing infrastructure)
  Trail Browsing's filter logic uses a useEffect tied to a derived state slice rather than the raw filter values, so consecutive filter changes can produce stale renders. The trail detail modal's outside-click close handler relies on event delegation that misfires when the modal renders above certain trail cards. The Calculator's money savings bar inherits the same proportional-scaling logic that caused Sprint 4's CO2 chart visual ambiguity. All three are fragile state patterns carried into Sprint 6.
- What part of your system is MOST likely to break during Sprint 6 integration?
  The Trail Browsing and the Calculator handoff is most likely to break during Sprint 6 Integration. Both features share a "selected mode" notion in different state shapes, and any Sprint 6 stabilization work that touches global state — particularly if a deferred feature like Personalized recs (#11) or Customize Routes for fitness levels (#15) is pulled in despite being out of scope — risks tipping this fragile cross-feature dependency into visible failure during the final demo.
- What workflow weakness still threatens final presentation quality?
  The Sprint 4 walkthrough rule, as we have implemented it, happens 5 minutes after class rather than inside class. That removes in-class observability which means that we cannot see a feature move back to In Progress in real time, and on days that we are absent (notably 5/13 and 5/14) the walkthrough is skipped.
- What feature or system should NOT be expanded further before feature freeze and why?
  Gamify app (#12) and Forum to talk (#7), both currently open as a part of our Product Backlog. Neither has merged code, and both require significant new state and UI surface, and pulling either into Sprint 6 would directly compete with the stabilization

work Trail Browsing's filter logic and modal close handler still need before feature freeze.

Section 6 – Sprint 6 Release Rule (STRICT FORMAT REQUIRED)

- You MUST use this format:
  At [time], [role] will [observable action].
  Your Rule:
  At the start of the final 15 minutes of every class period in Sprint 6 beginning Dev Day 1, Aarush Kapoor (Integration Verifier) will project a clean, logged-out browser on the classroom screen and run every Sprint Goal component back-to-back twice in a single session, and any feature that fails on the second consecutive run, requires verbal explanation, or shows visual inconsistency must be moved back to "In Progress" on GitHub before class ends.
- What specific release-readiness problem does this rule solve?
  (Must directly connect to content in Sections 2 – 5)
  This rule replaces the Sprint 4 walkthrough rule with a stricter, in-class-observable version that fixes the two structural gaps from Sections 2–5. The original rule was executed 5 minutes after class, removing in-class observability and making it skippable when class ended in a rush or members were absent; running it during the final 15 minutes of class brings it back inside the visible workflow the teacher and team can see in real time. The original rule required only one successful walkthrough pass, which let demo-only stability bugs like Trail Browsing's filter stale-state survive into our demo.
- What does this rule specifically improve or protect:
  (Check all that apply)
  ☒ Stability
  ☒ Repeatability
  ☒ Integration readiness
  ☐ Technical debt control
  ☐ Testing enforcement
  ☒ Workflow discipline
  ☒ Definition of Done enforcement
  ☒ Regression prevention
  ☒ Demo reliability

Section 7 – Enforcement System

- WHO is responsible for enforcement?
  Name + Role: **Aarush Kapor, Intergration verifier**
- WHEN is this enforced?
  (Specific moment only – not "throughout the Sprint")
  The final 15 minutes of every class period in Sprint 6 beginning Dev Day 1. Not five minutes after class; not before class. Inside the class period itself, on a different computer , during scheduled class time, directly correcting the Sprint 5 timing drift.

- HOW is this visibly checked during daily work?
  (What would someone SEE happening?)
  Aarush opens a fresh incognito browser tab on each of our computers on a different persons computer, logs in as a test account, and runs every Sprint Goal feature back-to-back twice in the same session.
- What happens IMMEDIATELY if this rule is violated?
  (Must define an actual interruption, restriction, rollback, or workflow consequence.)
  Aarush runs git revert on the offending feature's most recent merge to main during the same class period. The feature's GitHub ticket is reopened with the failure mode documented in the comments. No new merges to main are allowed in that class period until the revert is pushed. The developer who owned the reverted feature pairs with Aarush at the next class to write the fix.
- Escalation Scenario:
  If this rule is ignored for TWO consecutive class periods, what happens next?
  If the walkthrough is not run on two consecutive class periods (whether due to time pressure, absence, or skipping), Aarush blocks all merges to main until a missed walkthrough is completed on the next available class period. If a single feature fails the second-run check on three different class periods, that feature is cut from the Sprint 6 demo scope entirely and the team replans the demo around what is currently stable.
- Integration Protection Check:
  How does this enforcement system prevent unstable work from reaching Sprint 6 integration?
  Every feature reaching the Sprint 6 final demo will have passed at least one in-class, two-consecutive run walkthrough on the merged main branch. Anything that fails the second run gets reverted from main the same day, so unstable code physically cannot sit on main during Sprint 6 integration. The 5/13–5/14 pattern, walkthrough silently skipped during whole-team absences, is closed off because the escalation halts merges until a missed walkthrough catches up.

Section 8 – Measurable Indicator of Success
These must be observable and verifiable through: GitHub, Sprint board, testing behavior, integration results, and/or demo behavior

- Indicator 1:
  Zero Sprint Goal features remain in "Done" on the GitHub board at the end of any Sprint 6 class period without having passed Aarush's two consecutive-run walkthrough that day or a prior day verifiable by cross referencing the GitHub board move history against Aarush's walkthrough log.
- Indicator 2:
  Zero merges to main during Sprint 6 sit unverified for more than one class period verifiable by GitHub merge timestamps against the daily walkthrough log. Any merge older than one class period without a passing two run walkthrough triggers Aarush's automatic revert.

- Indicator 3:
  What measurable evidence will prove the system works repeatedly and reliably?
  Every feature in the Sprint 6 demo passes Aarush's two consecutive-run walkthrough on at least two different class periods before the demo, on a clean logged-out browser, with no rollback or revert between those passes verifiable by GitHub board history showing two separate walkthrough confirmed done states per demo feature.

Section 9 – Sprint 6 Feature Freeze Readiness

- Is your current system ready for feature freeze?
  Partially
- What must still be stabilized **BEFORE** feature freeze?
  (1) Trail Browsing's filter logic, so consecutive filter changes produce a clean re-render of the trail list without manual refresh. (2) The trail detail modal's outside-click close handler, so closing behavior is consistent regardless of which trail card is opened. (3) The Calculator's money savings bar proportional scaling, to close the visual-ambiguity issue carried over from Sprint 4's CO2 chart. (4) A documented end-to-end happy-path script that exercises Trail Browsing to calculator to activity log in sequence and can be run by anyone in less than 5 minutes.
- What process behavior **MUST** improve immediately in Sprint 6?
  The walkthrough must move back inside class time and must require two consecutive runs on the same browser session. One pass and done, executed five minutes after class, is the structural pattern that produced the Sprint 5 demo failure and must end at Sprint 6 Dev Day 1.
- What is the single biggest remaining release risk?
  Trail Browsing's filter logic failing under repeated use during the final demo. Trail Browsing was identified in the Sprint 5 plan as the highest-risk item, its filter bug is still un-refactored heading into the feature-freeze sprint, and it sits at the center of Momentum's planning-tool proposition.

Section 10 – Rule Validation Checklist

Your rule is only valid if ALL boxes can be checked:

☒ This rule can be observed during class
☒ This rule has a specific owner
☒ This rule happens at a clearly defined time
☒ This rule would visibly fail if ignored
☒ This rule directly prevents this Sprint's failure
☒ This rule is enforced BEFORE demo day
☒ This rule prevents unstable work from reaching integration
☒ This rule creates measurable release-readiness improvement

If any box cannot be checked, your rule is not valid and must be revised.